Anthropic Courses EN

Anthropic

Table of contents

1 Anthropic courses

Welcome to Anthropic's educational courses. This repository currently contains five courses. We suggest completing the courses in the following order:

1. Anthropic API fundamentals - teaches the essentials of working with the Claude SDK: getting an API key, working with model parameters, writing multimodal prompts, streaming responses, etc.
2. Prompt engineering interactive tutorial - a comprehensive step-by-step guide to key prompting techniques. [AWS Workshop version]
3. Real world prompting - learn how to incorporate prompting techniques into complex, real world prompts. [Google Vertex version]
4. Prompt evaluations - learn how to write production prompt evaluations to measure the quality of your prompts.
5. Tool use - teaches everything you need to know to implement tool use successfully in your workflows with Claude.

Please note that these courses often favor our lowest-cost model, Claude 3 Haiku, to keep API costs down for students following along with the materials. Feel free to use other Claude models if you prefer.

Part I

1. Anthropic API Fundamentals

2 Anthropic API fundamentals

A series of notebook tutorials that cover the essentials of working with Claude models and the Anthropic SDK including:

- Getting an API key and making simple requests
- Working with the messages format
- Comparing capabilities and performance of the Claude model family
- Understanding model parameters
- Working with streaming responses
- Vision prompting

3 Getting started with the Claude SDK

3.1 Lesson goals

In this first lesson, you'll learn how to: * install the necessary packages and authenticate with the API * make your first request to the Claude AI assistant

3.2 Installing the SDK

Before diving into the SDK, make sure you have Python installed on your system.

The Claude Python SDK requires Python 3.7.1 or later.

You can check your current Python version by running the following command in your terminal:

```
python --version
```

If you don't have Python installed or your version is older than 3.7.1, please visit the official Python website and follow the installation instructions for your operating system.

With Python ready, you can now install the Anthropic package using pip

```
# Use this command if installing the package from inside a notebook
%pip install anthropic
#Use this command to install the package from the command line
# pip install anthropic
```

3.3 Getting an API key

To authenticate your requests to the Claude API, you'll need an API key.

Follow these steps to obtain your API key:

1. If you haven't already, sign up for an Anthropic account by visiting https://console.anthropic.com
2. Once you've created your account and logged in, navigate to the API settings page. You can find this page by clicking on your profile icon in the top-right corner and selecting "API Keys" from the dropdown menu, or by navigating to the "API Keys" menu in the Settings tab.
3. On the API settings page, click on the "Create Key" button. A modal window will appear, prompting you to give your key a descriptive name. Choose a name that reflects the purpose or project you'll be using the key for. You can create as many keys as you want within your account (note that rate and message limits apply at the account level, not the API key level).
4. After entering a name, click on the "Create" button. Your new API key will be generated and displayed on the screen. > Make sure to copy this key, as you won't be able to view it again once you navigate away from this page.

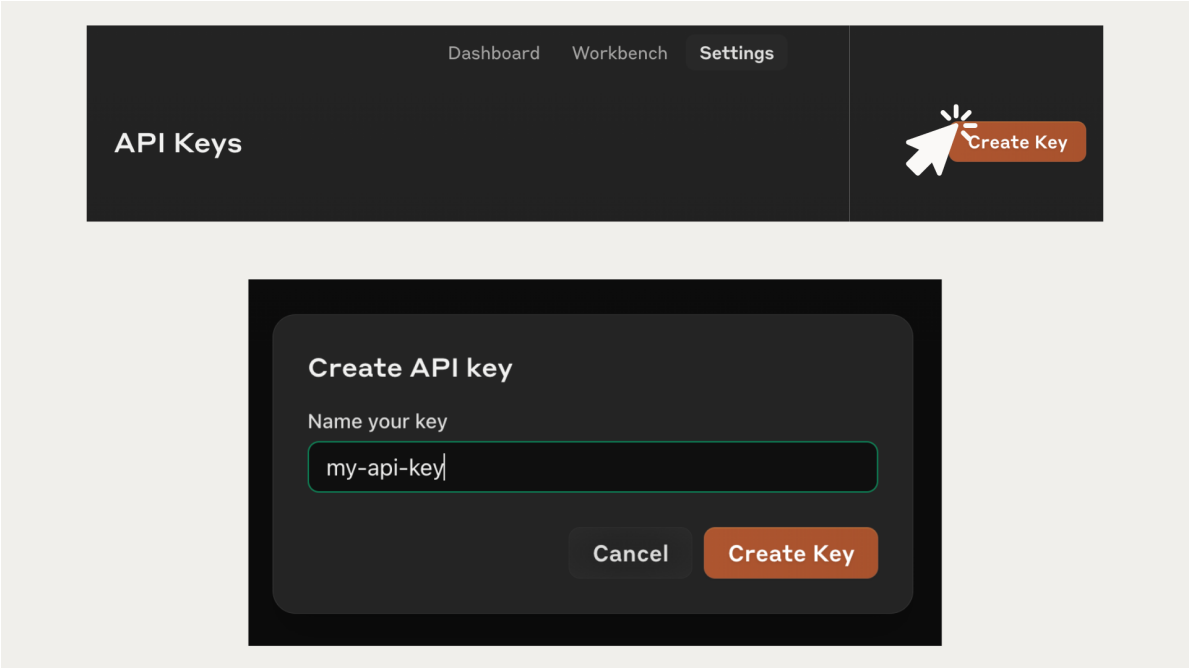

Figure 3.1: signup.png

Remember, your API key is a sensitive piece of information that grants access to your An-

thropic account. Treat it like a password and never share it publicly or commit it to version control systems like Git.

3.4 Safely storing your API key

While you can hardcode your API key directly in your Python scripts, it's generally considered best practice to keep sensitive information, like API keys, separate from your codebase. One common approach is to store the API key in a `.env` file and load it using the `python-dotenv package`. Here's how you can set it up:

Create a new file called `.env` in the same directory as your notebook.

Add your API key to the newly created `.env` file using the following format:

```
ANTHROPIC_API_KEY=put-your-api-key-here
```

Make sure to save the `.env` file

Install the python-dotenv package by running the following command in your terminal or notebook:

```
#To install the package from a notebook:
%pip install python-dotenv

#To install the package from your terminal:
# pip install python-dotenv
```

We can now load the API key from the .env file using the `load_dotenv()` function from the dotenv module with the following code:

```
from dotenv import load_dotenv
import os

load_dotenv()
my_api_key = os.getenv("ANTHROPIC_API_KEY")
```

3.5 Making basic requests with the client

With the `anthropic` package installed and your API key loaded, you're ready to start making requests to the Claude API.

The first step is to create a client object, which serves as the main entry point for interacting with the API.

```
from anthropic import Anthropic

client = Anthropic(
    api_key=my_api_key
)
```

Note that the `anthropic` SDK automatically looks for an environment variable called "ANTHROPIC_API_KEY", so you don't actually have to pass it in manually and can instead just do this:

```
from anthropic import Anthropic

client = Anthropic()
```

Now that we have our client instantiated, it's time to make our first request.

To send a message to Claude and receive a response, we'll use the `messages.create()` method of the `client` object. We'll talk about the specific parameters and response format in the next lesson. For now, try running the following code and you should get your first message back from Claude!

```
our_first_message = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=1000,
    messages=[
        {"role": "user", "content": "Hi there! Please write me a haiku about a pet chicken"}
    ]
)

print(our_first_message.content[0].text)
```

```
Feathered friend clucking,
Scratching in the dirt all day,
Loyal pet chicken.
```

3.6 Exercise

We've only just begun, so this exercise might feel a little underwhelming. It's always good to get some practice with the basics.

Please do the following: 1. Create a new notebook or Python script. 2. Import the proper packages 3. Load your Anthropic API key 4. Ask Claude to tell you a joke and then print out the result (you can copy/paste the code above and tweak it)

---

4 Working with messages

4.1 Lesson goals

- Understand the messages API format
- Work with and understand model response objects
- Build a simple multi-turn chatbot

4.2 Basic setup

We'll start by importing the packages we need and initializing a client object. See the previous tutorial for details on how to get an API key and properly store it.

```
from dotenv import load_dotenv
from anthropic import Anthropic

#load environment variable
load_dotenv()

#automatically looks for an "ANTHROPIC_API_KEY" environment variable
client = Anthropic()
```

4.3 Messages format

As we saw in the previous lesson, we can use `client.messages.create()` to send a message to Claude and get a response:

```
response = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=1000,
    messages=[
        {"role": "user", "content": "What flavors are used in Dr. Pepper?"}
    ]
```

```
)

print(response)
```

```
Message(id='msg_013wVsHLHRjuDM2WgvVJ8RNm', content=[ContentBlock(text='The exact flavor formu
```

Let's take a closer look at this bit:

```
messages=[
        {"role": "user", "content": "What flavors are used in Dr. Pepper?"}
    ]
```

The messages parameter is a crucial part of interacting with the Claude API. It allows you to provide the conversation history and context for Claude to generate a relevant response.

The messages parameter expects a list of message dictionaries, where each dictionary represents a single message in the conversation. Each message dictionary should have the following keys:

- `role`: A string indicating the role of the message sender. It can be either "user" (for messages sent by the user) or "assistant" (for messages sent by Claude).
- `content`: A string or list of content dictionaries representing the actual content of the message. If a string is provided, it will be treated as a single text content block. If a list of content dictionaries is provided, each dictionary should have a "type" (e.g., "text" or "image") and the corresponding content. For now, we'll leave `content` as a single string.

Here's an example of a messages list with a single user message:

```
messages = [
    {"role": "user", "content": "Hello Claude! How are you today?"}
]
```

And here's an example with multiple messages representing a conversation:

```
messages = [
    {"role": "user", "content": "Hello Claude! How are you today?"},
    {"role": "assistant", "content": "Hello! I'm doing well, thank you. How can I assist you
    {"role": "user", "content": "Can you tell me a fun fact about ferrets?"},
    {"role": "assistant", "content": "Sure! Did you know that excited ferrets make a clucking
]
```

Remember that messages always alternate between user and assistant messages.

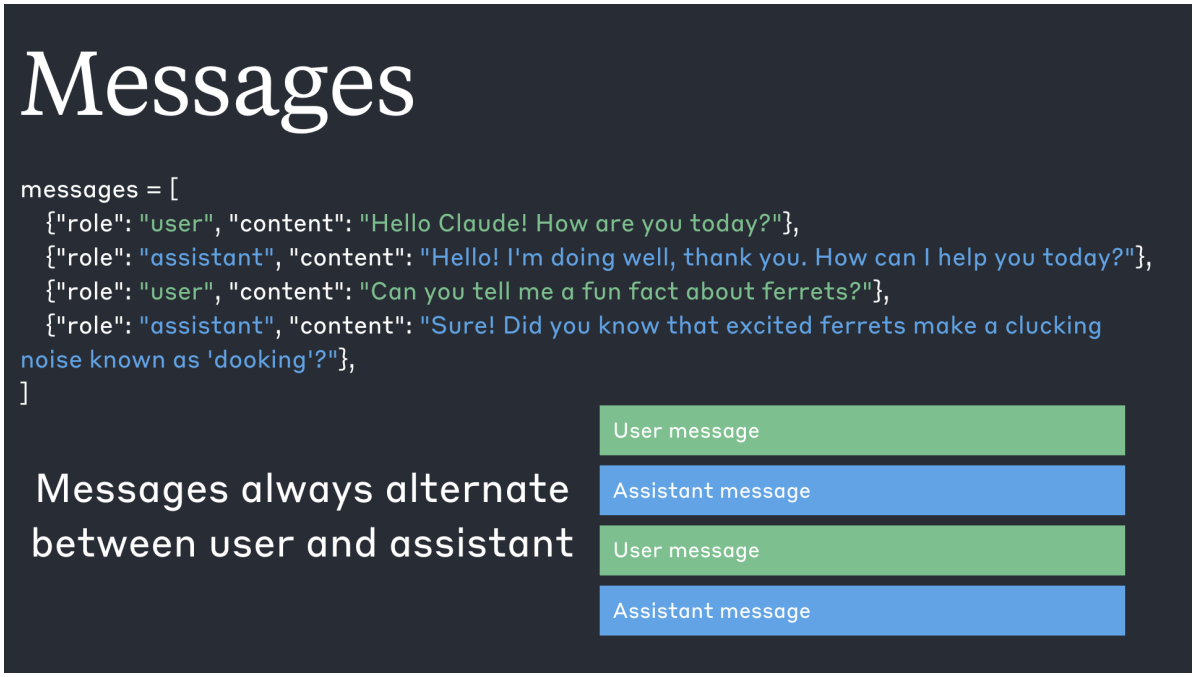

Figure 4.1: alternating_messages.png

The messages format allows us to structure our API calls to Claude in the form of a conversation, allowing for **context preservation**: The messages format allows for maintaining an entire conversation history, including both user and assistant messages. This ensures that Claude has access to the full context of the conversation when generating responses, leading to more coherent and relevant outputs.

Note: many use-cases don't require a conversation history, and there's nothing wrong with providing a list of messages that only contains a single message!

4.4 Quiz

What are the two required keys in each message?

- **a)** "sender" and "text"
- **b)** "role" and "content"
- **c)** "user" and "assistant"

- **d)** “input” and “output”

View quiz answer

The correct answer is b. Every message should have a “role” and “content”

4.5 Inspecting the message response

Next, let’s take a look at the shape of the response we get back from Claude.

Let’s ask Claude to do something simple:

```
response = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=1000,
    messages=[
        {"role": "user", "content": "Translate hello to French. Respond with a single word"}
    ]
)
```

Now let’s inspect the contents of the `response` that we get back:

```
response
```

```
Message(id='msg_01SuDqJSTJaRpkDmHGrbfxCt', content=[ContentBlock(text='Bonjour.', type='text
```

We get back a `Message` object that contains a handful of properties. Here’s an example:

```
Message(id='msg_01Mq5gDnUmDESukTgwPV8xtG', content=[TextBlock(text='Bonjour.', type='text')]
```

The most important piece of information is the `content` property: this contains the actual content the model generated for us. This is a **list** of content blocks, each of which has a type that determines its shape.

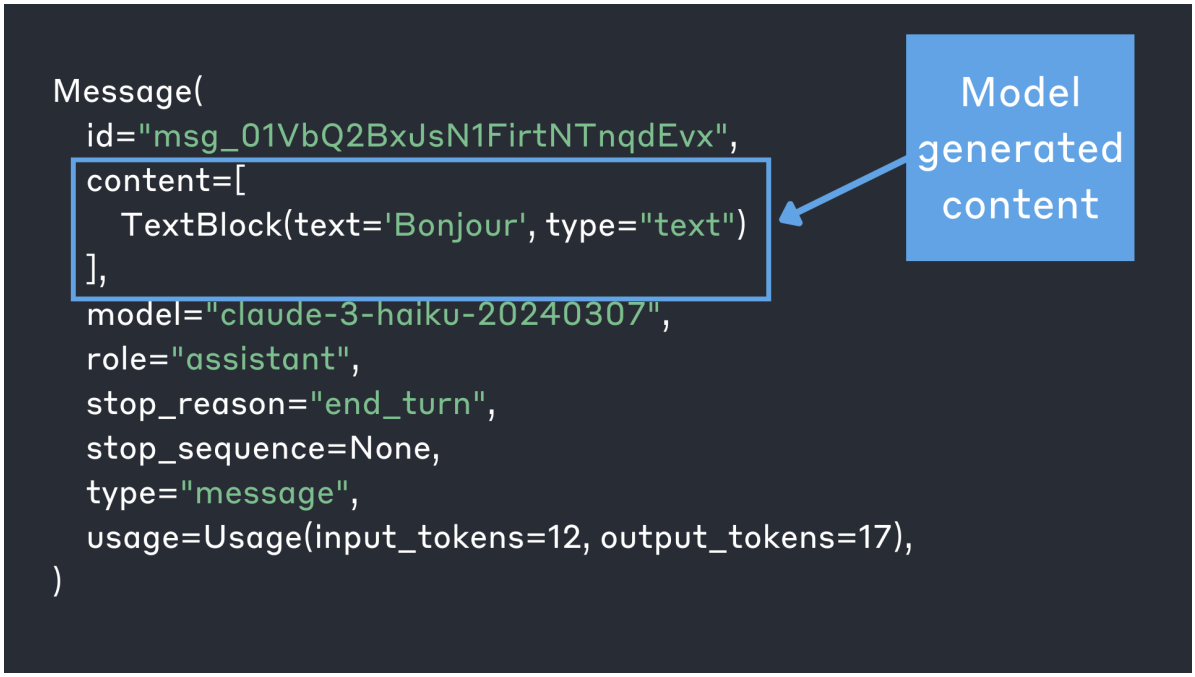

Figure 4.2: message_content.png

In order to access the actual text content of the model's response, we need to do the following:

```
print(response.content[0].text)
```

```
Bonjour.
```

In addition to `content`, the `Message` object contains some other pieces of information:

- `id` - a unique object identifier
- `type` - The object type, which will always be "message"
- `role` - The conversational role of the generated message. This will always be "assistant".
- `model` - The model that handled the request and generated the response
- `stop_reason` - The reason the model stopped generating. We'll learn more about this later.
- `stop_sequence` - We'll learn more about this shortly.
- `usage` - information on billing and rate-limit usage. Contains information on:
  - `input_tokens` - The number of input tokens that were used.
  - `output_tokens` - The number of output tokens that were used.

It's important to know that we have access to these pieces of information, but if you only remember one thing, make it this: `content` contains the actual model-generated content

---

4.6 Exercise

Write a function called translate that expects two arguments: * A word * A language

When you call the `translate` function, it should return the result of asking Claude to translate `word` into `language`. For example:

```
translate("hello", "Spanish")
# 'The word "hello" translated into Spanish is: Hola'

translate("chicken", "Italian")
# 'The Italian word for "chicken" is: pollo'
```

Bonus points if you can write a prompt so that Claude only responds with the translated word and no preamble, like this:

```
translate("chicken", "Italian")
# 'pollo'
```

View exercise solution

Here's one possible solution:

```
  def translate(word, language):
    response = client.messages.create(
        model="claude-3-opus-20240229",
        max_tokens=1000,
        messages=[
            {"role": "user", "content": f"Translate the word {word} into {language}.  Only re
        ]
    )
    return response.content[0].text
```

4.7 Message list mistakes

4.7.1 Mistake #1: starting with an assistant message

When you're starting out, it's easy to make mistakes when working with the `messages` list. The list of messages must start with a `user` message. The following code generates an error because the messages list starts with an assistant message:

```
response = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=1000,
    messages=[
        {"role": "assistant", "content": "Hello there!"}
    ]
)
print(response.content[0].text)
```

```
BadRequestError: Error code: 400 - {'type': 'error', 'error': {'type': 'invalid_request_error
---------------------------------------------------------------------------
BadRequestError                           Traceback (most recent call last)
Cell In[10], line 1
----> 1 response = client.messages.create(
      2     model="claude-3-haiku-20240307",
      3     max_tokens=1000,
      4     messages=[
      5         {"role": "assistant", "content": "Translate hello to French. Respond with a s
      6     ]
      7 )
      8 print(response.content[0].text)
File /opt/homebrew/Caskroom/miniforge/base/envs/py311/lib/python3.11/site-packages/anthropic/
    275             msg = f"Missing required argument: {quote(missing[0])}"
    276         raise TypeError(msg)
--> 277 return func(*args, **kwargs)
File /opt/homebrew/Caskroom/miniforge/base/envs/py311/lib/python3.11/site-packages/anthropic/
    650 @required_args(["max_tokens", "messages", "model"], ["max_tokens", "messages", "model
    651 def create(
    652     self,
   (...)
    679     timeout: float | httpx.Timeout | None | NotGiven = 600,
    680 ) -> Message | Stream[MessageStreamEvent]:
--> 681     return self._post(
    682         "/v1/messages",
```

```
    683         body=maybe_transform(
    684             {
    685                 "max_tokens": max_tokens,
    686                 "messages": messages,
    687                 "model": model,
    688                 "metadata": metadata,
    689                 "stop_sequences": stop_sequences,
    690                 "stream": stream,
    691                 "system": system,
    692                 "temperature": temperature,
    693                 "top_k": top_k,
    694                 "top_p": top_p,
    695             },
    696             message_create_params.MessageCreateParams,
    697         ),
    698         options=make_request_options(
    699             extra_headers=extra_headers, extra_query=extra_query, extra_body=extra_bo
    700         ),
    701         cast_to=Message,
    702         stream=stream or False,
    703         stream_cls=Stream[MessageStreamEvent],
    704     )
File /opt/homebrew/Caskroom/miniforge/base/envs/py311/lib/python3.11/site-packages/anthropic/
   1218 def post(
   1219     self,
   1220     path: str,
   (...)
   1227     stream_cls: type[_StreamT] | None = None,
   1228 ) -> ResponseT | _StreamT:
   1229     opts = FinalRequestOptions.construct(
   1230         method="post", url=path, json_data=body, files=to_httpx_files(files), **optio
   1231     )
-> 1232     return cast(ResponseT, self.request(cast_to, opts, stream=stream, stream_cls=stre
File /opt/homebrew/Caskroom/miniforge/base/envs/py311/lib/python3.11/site-packages/anthropic/
    912 def request(
    913     self,
    914     cast_to: Type[ResponseT],
   (...)
    919     stream_cls: type[_StreamT] | None = None,
    920 ) -> ResponseT | _StreamT:
--> 921     return self._request(
    922         cast_to=cast_to,
    923         options=options,
```

```
    924         stream=stream,
    925         stream_cls=stream_cls,
    926         remaining_retries=remaining_retries,
    927     )
File /opt/homebrew/Caskroom/miniforge/base/envs/py311/lib/python3.11/site-packages/anthropic/
   1009         err.response.read()
   1011     log.debug("Re-raising status error")
-> 1012     raise self._make_status_error_from_response(err.response) from None
   1014 return self._process_response(
   1015     cast_to=cast_to,
   1016     options=options,
   (...)
   1019     stream_cls=stream_cls,
   1020 )
BadRequestError: Error code: 400 - {'type': 'error', 'error': {'type': 'invalid_request_error
```

4.7.2 Mistake #2: improperly alternating messages

Messages must alternate between `user` and `assistant`, and we'll get an error if we don't follow this rule:

```
response = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=1000,
    messages=[
        {"role": "user", "content": "Hey there!"},
        {"role": "assistant", "content": "Hi there!"},
        {"role": "assistant", "content": "How can I help you??"}
    ]
)
print(response.content[0].text)
```

```
BadRequestError: Error code: 400 - {'type': 'error', 'error': {'type': 'invalid_request_error
---------------------------------------------------------------------------
BadRequestError                           Traceback (most recent call last)
Cell In[12], line 1
----> 1 response = client.messages.create(
      2     model="claude-3-haiku-20240307",
      3     max_tokens=1000,
      4     messages=[
      5         {"role": "user", "content": "Hey there!"},
      6         {"role": "assistant", "content": "Hi there!"},
```

```
      7         {"role": "assistant", "content": "How can I help you??"}
      8     ]
      9 )
     10 print(response.content[0].text)
File /opt/homebrew/Caskroom/miniforge/base/envs/py311/lib/python3.11/site-packages/anthropic/
    275             msg = f"Missing required argument: {quote(missing[0])}"
    276     raise TypeError(msg)
--> 277 return func(*args, **kwargs)
File /opt/homebrew/Caskroom/miniforge/base/envs/py311/lib/python3.11/site-packages/anthropic/
    650 @required_args(["max_tokens", "messages", "model"], ["max_tokens", "messages", "model
    651 def create(
    652     self,
   (...)
    679     timeout: float | httpx.Timeout | None | NotGiven = 600,
    680 ) -> Message | Stream[MessageStreamEvent]:
--> 681     return self._post(
    682         "/v1/messages",
    683         body=maybe_transform(
    684             {
    685                 "max_tokens": max_tokens,
    686                 "messages": messages,
    687                 "model": model,
    688                 "metadata": metadata,
    689                 "stop_sequences": stop_sequences,
    690                 "stream": stream,
    691                 "system": system,
    692                 "temperature": temperature,
    693                 "top_k": top_k,
    694                 "top_p": top_p,
    695             },
    696             message_create_params.MessageCreateParams,
    697         ),
    698         options=make_request_options(
    699             extra_headers=extra_headers, extra_query=extra_query, extra_body=extra_bo
    700         ),
    701         cast_to=Message,
    702         stream=stream or False,
    703         stream_cls=Stream[MessageStreamEvent],
    704     )
File /opt/homebrew/Caskroom/miniforge/base/envs/py311/lib/python3.11/site-packages/anthropic/
   1218 def post(
   1219     self,
   1220     path: str,
```

```
   (...)
   1227     stream_cls: type[_StreamT] | None = None,
   1228 ) -> ResponseT | _StreamT:
   1229     opts = FinalRequestOptions.construct(
   1230         method="post", url=path, json_data=body, files=to_httpx_files(files), **optic
   1231     )
-> 1232     return cast(ResponseT, self.request(cast_to, opts, stream=stream, stream_cls=stre
File /opt/homebrew/Caskroom/miniforge/base/envs/py311/lib/python3.11/site-packages/anthropic/
    912 def request(
    913     self,
    914     cast_to: Type[ResponseT],
   (...)
    919     stream_cls: type[_StreamT] | None = None,
    920 ) -> ResponseT | _StreamT:
--> 921     return self._request(
    922         cast_to=cast_to,
    923         options=options,
    924         stream=stream,
    925         stream_cls=stream_cls,
    926         remaining_retries=remaining_retries,
    927     )
File /opt/homebrew/Caskroom/miniforge/base/envs/py311/lib/python3.11/site-packages/anthropic/
   1009         err.response.read()
   1011     log.debug("Re-raising status error")
-> 1012     raise self._make_status_error_from_response(err.response) from None
   1014 return self._process_response(
   1015     cast_to=cast_to,
   1016     options=options,
   (...)
   1019     stream_cls=stream_cls,
   1020 )
BadRequestError: Error code: 400 - {'type': 'error', 'error': {'type': 'invalid_request_erro
```

4.8 Messages list use cases

4.8.1 Putting words in Claude's mouth

Another common strategy for getting very specific outputs is to "put words in Claude's mouth". Instead of only providing `user` messages to Claude, we can also supply an `assistant` message that Claude will use when generating output.

When using Anthropic's API, you are not limited to just the `user` message. If you supply an `assistant` message, Claude will continue the conversation from the last `assistant` token. Just remember that we must start with a `user` message.

Suppose I want Claude to write me a haiku that starts with the first line, "calming mountain air". I can provide the following conversation history:

```
messages=[
        {"role": "user", "content": f"Generate a beautiful haiku"},
        {"role": "assistant", "content": "calming mountain air"}
    ]
```

We tell Claude that we want it to generate a Haiku AND we put the first line of the Haiku in Claude's mouth

```
response = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=500,
    messages=[
        {"role": "user", "content": f"Generate a beautiful haiku"},
        {"role": "assistant", "content": "calming mountain air"}
    ]
)
print(response.content[0].text)
```

```
,
dancing sunlight on still waters,
nature's gentle grace.
```

To get the entire haiku, starting with the line we provided:

```
print("calming mountain air" + response.content[0].text)
```

```
calming mountain air,
dancing sunlight on still waters,
nature's gentle grace.
```

4.8.2 Few-shot prompting

One of the most useful prompting strategies is called "few-shot prompting" which involves providing a model with a small number of **examples**. These examples help guide Claude's generated output. The messages conversation history is an easy way to provide examples to Claude.

For example, suppose we want to use Claude to analyze the sentiment in tweets. We could start by simply asking Claude to "please analyze the sentiment in this tweet:" and see what sort of output we get:

```
response = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=500,
    messages=[
        {"role": "user", "content": f"Analyze the sentiment in this tweet: Just tried the new
    ]
)
print(response.content[0].text)
```

```
The sentiment in this tweet is overwhelmingly positive. The user expresses their enjoyment of

Positive indicators:
1. "My taste buds are doing a happy dance!" - This phrase indicates that the user is extremel

2. Emojis - The use of the hot pepper  and cucumber  emojis further emphasizes the user's ex

3. Hashtags - The inclusion of #pickleslove and #spicyfood hashtags suggests that the user ha

4. Exclamation mark - The exclamation mark at the end of the first sentence adds emphasis to

Overall, the tweet conveys a strong sense of satisfaction, excitement, and enjoyment related
```

The first time I ran the above code, Claude generated this long response:

```
The sentiment in this tweet is overwhelmingly positive. The user expresses their enjoyment of

Positive indicators:
1. "My taste buds are doing a happy dance!" - This phrase indicates that the user is extremel

2. Emojis - The use of the hot pepper  and cucumber  emojis further emphasizes the user's ex
```

```
3. Hashtags - The inclusion of #pickleslove and #spicyfood hashtags suggests that the user ha

4. Exclamation mark - The exclamation mark at the end of the first sentence adds emphasis to

Overall, the tweet conveys a strong sense of satisfaction, excitement, and enjoyment related
```

This is a great response, but it's probably way more information than we need from Claude, especially if we're trying to automate the sentiment analysis of a large number of tweets.

We might prefer that Claude respond with a standardized output format like a single word (POSITIVE, NEUTRAL, NEGATIVE) or a numeric value (1, 0, -1). For readability and simplicity, let's get Claude to respond with either "POSITIVE" or "NEGATIVE". One way of doing this is through few-shot prompting. We can provide Claude with a conversation history that shows exactly how we want it to respond:

```
messages=[
        {"role": "user", "content": "Unpopular opinion: Pickles are disgusting. Don't @ me"}
        {"role": "assistant", "content": "NEGATIVE"},
        {"role": "user", "content": "I think my love for pickles might be getting out of hand
        {"role": "assistant", "content": "POSITIVE"},
        {"role": "user", "content": "Seriously why would anyone ever eat a pickle?  Those thi
        {"role": "assistant", "content": "NEGATIVE"},
        {"role": "user", "content": "Just tried the new spicy pickles from @PickleCo, and my
    ]
```

```
response = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=500,
    messages=[
        {"role": "user", "content": "Unpopular opinion: Pickles are disgusting. Don't @ me"}
        {"role": "assistant", "content": "NEGATIVE"},
        {"role": "user", "content": "I think my love for pickles might be getting out of hand
        {"role": "assistant", "content": "POSITIVE"},
        {"role": "user", "content": "Seriously why would anyone ever eat a pickle?  Those thi
        {"role": "assistant", "content": "NEGATIVE"},
        {"role": "user", "content": "Just tried the new spicy pickles from @PickleCo, and my
    ]
)
print(response.content[0].text)
```

```
POSITIVE
```

4.9 Exercise

4.9.1 Your task: build a chatbot

Build a simple multi-turn command-line chatbot script. The messages format lends itself to building chat-based applications. To build a chat-bot with Claude, it's as simple as:

1. Keep a list to store the conversation history
2. Ask a user for a message using `input()` and add the user input to the messages list
3. Send the message history to Claude
4. Print out Claude's response to the user
5. Add Claude's assistant response to the history
6. Go back to step 2 and repeat! (use a loop and provide a way for users to quit!)

View exercise solution

```
    conversation_history = []

    while True:
        user_input = input("User: ")

        if user_input.lower() == "quit":
            print("Conversation ended.")
            break

        conversation_history.append({"role": "user", "content": user_input})

        response = client.messages.create(
            model="claude-3-haiku-20240307",
            messages=conversation_history,
            max_tokens=500
        )

        assistant_response = response.content[0].text
        print(f"Assistant: {assistant_response}")
        conversation_history.append({"role": "assistant", "content": assistant_response})
    ```
</details>

***
```
```

````
`<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImFudGhyb3BpY19hcGlfZnVuZGFtZW50YWxzIn0= -->

```{=html}
<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImFudGhyb3BpY19hcGlfZnVuZGFtZW50YWxzIiwiYm9va
````
````

5 Models

5.1 Lesson Goals

- Understand the various Claude models
- Compare the speed and capabilities of the Claude models

Let's start by importing the `anthropic` SDK and loading our API key:

```
from dotenv import load_dotenv
from anthropic import Anthropic

load_dotenv()

client = Anthropic()
```

5.2 Claude models

The Claude Python SDK supports multiple models, each with different capabilities and performance characteristics. This visualization compares cost vs. speed across Claude 3 and 3.5 models, showcasing the range in tradeoffs between cost and intelligence:

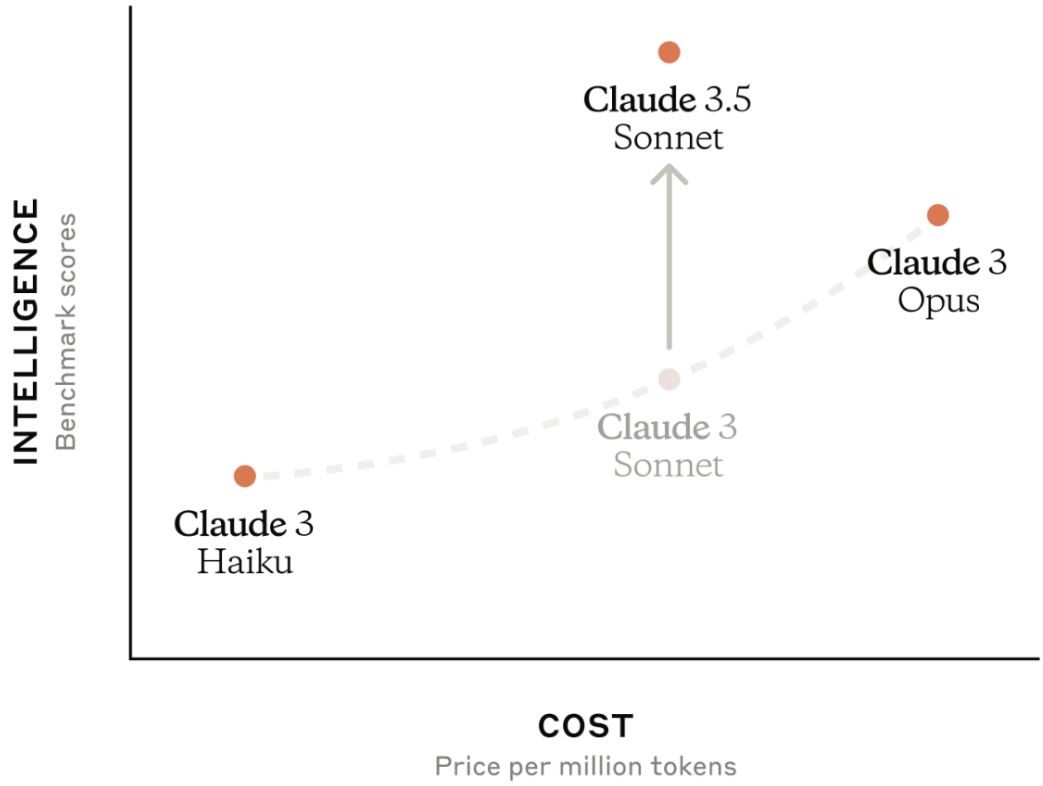

Figure 5.1: models.png

When choosing a model, there a few important factors to consider:

- The model's latency (how fast is it?)
- The model's capabilities (how smart is it?)
- The model's cost (how expensive is it?)

Refer to this table for a comparison of the key features and capabilities of each model in the Claude family.

5.3 Comparing model speeds

Below is a simple function that runs the same prompt for all 4 models and prints out the model response and the time each request took.

```python
import time
def compare_model_speeds():
    models = ["claude-3-5-sonnet-20240620","claude-3-opus-20240229", "claude-3-sonnet-202402
    task = "Explain the concept of photosynthesis in a concise paragraph."

    for model in models:
        start_time = time.time()

        response = client.messages.create(
            model=model,
            max_tokens=500,
            messages=[{"role": "user", "content": task}]
        )

        end_time = time.time()
        execution_time = end_time - start_time
        tokens = response.usage.output_tokens
        time_per_token = execution_time/tokens

        print(f"Model: {model}")
        print(f"Response: {response.content[0].text}")
        print(f"Generated Tokens: {tokens}")
        print(f"Execution Time: {execution_time:.2f} seconds")
        print(f"Time Per Token: {time_per_token:.2f} seconds\n")
```

```python
compare_model_speeds()
```

```
Model: claude-3-5-sonnet-20240620
Response: Photosynthesis is the process by which plants, algae, and some bacteria convert lig
Generated Tokens: 146
Execution Time: 2.56 seconds
Time Per Token: 0.02 seconds

Model: claude-3-opus-20240229
Response: Photosynthesis is the process by which green plants and some other organisms use su
Generated Tokens: 146
Execution Time: 7.32 seconds
Time Per Token: 0.05 seconds

Model: claude-3-sonnet-20240229
Response: Photosynthesis is the process by which plants and certain other organisms convert l
```

```
Generated Tokens: 108
Execution Time: 2.64 seconds
Time Per Token: 0.02 seconds

Model: claude-3-haiku-20240307
Response: Photosynthesis is the process by which plants, algae, and some bacteria convert lig
Generated Tokens: 126
Execution Time: 1.09 seconds
Time Per Token: 0.01 seconds
```

The exact responses you get when running the above code will differ, but here's a table summarizing the particular output we got when running the above code:

Model	Generated Tokens	Execution Time (seconds)	Time Per Token (seconds)
claude-3-5-sonnet-20240620	146	2.56	0.02
claude-3-opus-20240229	146	7.32	0.05
claude-3-sonnet-20240229	108	2.64	0.02
claude-3-haiku-20240307	126	1.09	0.01

It's also important to note that with a simple prompt like "Explain the concept of photosynthesis in a concise paragraph," all of the models perform well. In this particular case, it would likely make sense to pick the fastest and cheapest option.

The above example is a simple illustration of model speed differences, but it's not a very rigorous demonstration. Here's a plot we generated by providing all 3 models the same input prompt 50 times and averaging the response time for each model. To ensure a "fair" comparison, we prompted the models to generate extremely long outputs, and then we cut off all model responses at exactly the same number of tokens using `max_tokens` (we cover this in the next lesson).

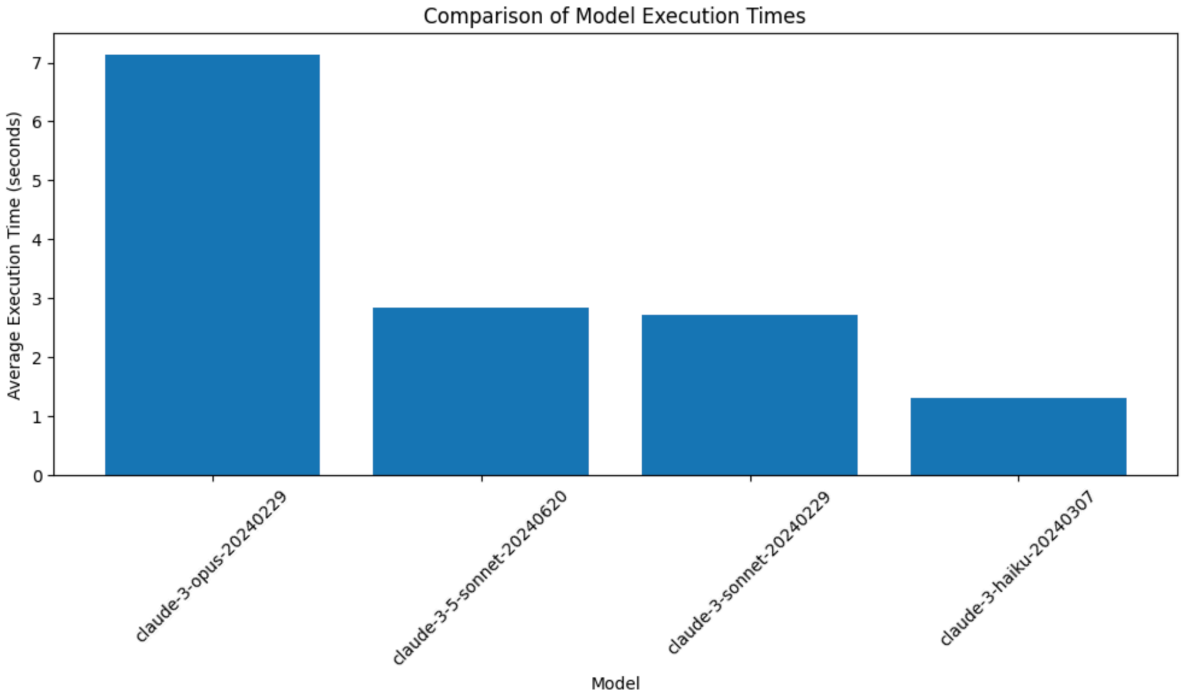

Figure 5.2: speed_comparison.png

5.4 Comparing model capabilities

Clearly Haiku is the fastest model, so why would we bother to use the others? It all comes down to the trade-off between model speed, cost, and overall capabilities. Haiku is the fastest, but its outputs may not be as high-quality as Opus's in certain situations. With that said, it's important to note that in many cases, Haiku can perform as well as some of our more capable models. The only way to truly know which model is the "best" for your specific use case is to try them out and evaluate their performance.

In general, we recommend using our most capable model, Claude 3.5 Sonnet, for use cases involving: * **Coding:** Claude 3.5 Sonnet writes, edits, and runs code autonomously, streamlining code translations for faster, more accurate updates and migrations. * **Customer support:** Claude 3.5 Sonnet understands user context and orchestrates multi-step workflows, enabling 24/7 support, faster responses, and improved customer satisfaction. * **Data science & analysis:** Claude 3.5 Sonnet navigates unstructured data, generates insights, and produces visualizations and predictions to enhance data science expertise. * **Visual processing:** Claude 3.5 Sonnet excels at interpreting charts, graphs, and images, accurately transcribing text to derive insights beyond just the text alone. * **Writing:** Claude 3.5 Sonnet represents a significant improvement in understanding nuance and humor, producing high-quality, authentic,

and relatable content.

If your intersted in a benchmark comparison between our Claude family of models, please read our Claude family model card for more information.

5.4.1 Demonstrating capabilities

It's hard to showcase the various capabilities of each model with a single demo, but the function below attempts to do so. We ask each of the three models to solve the following math problem:

```
What is the geometric monthly fecal coliform mean of a distribution system with the following
 counts: 24, 15, 7, 16, 31 and 23? The result will be inputted into a NPDES DMR, therefore, r
 to the nearest whole number
```

NOTE: the correct answer is 18

We ask each model to solve the math problem 7 times and record the answer each time:

```python
def compare_model_capabilities():
    models = ["claude-3-5-sonnet-20240620", "claude-3-opus-20240229", "claude-3-sonnet-20240
    task = """
    What is the geometric monthly fecal coliform mean of a distribution system with the follo
 counts: 24, 15, 7, 16, 31 and 23? The result will be inputted into a NPDES DMR, therefore, r
 to the nearest whole number.  Respond only with a number and nothing else.
    """

    for model in models:
        answers = []
        for attempt in range(7):
            response = client.messages.create(
                model=model,
                max_tokens=1000,
                messages=[{"role": "user", "content": task}]
            )
            answers.append(response.content[0].text)

        print(f"Model: {model}")
        print(f"Answers: ", answers)
```

```
compare_model_capabilities()
```

```
Model: claude-3-5-sonnet-20240620
Answers:  ['18', '18', '18', '18', '18', '18', '18']
Model: claude-3-opus-20240229
Answers:  ['18', '18', '18', '18', '18', '18', '18']
Model: claude-3-sonnet-20240229
Answers:  ['18', '17', '16', '17', '19', '18', '18']
Model: claude-3-haiku-20240307
Answers:  ['17', '17', '18', '17', '17', '17', '18']
```

The exact outputs we get from each model will vary, but here's a summary of the results from a single run:

- `claude-3-5-sonnet-20240620` - gets the right answer **7/7** times
- `claude-3-opus-20240229` - gets the right answer **7/7** times
- `claude-3-sonnet-20240229` - gets the right answer **3/7** times
- `claude-3-haiku-20240307` - gets the right answer **2/7** times

Clearly Claude 3.5 Sonnet and Claude 3 Opus perform best on this particular math problem.

Note: this is a very simplistic demonstration of model capabilities. It is not at all a rigorous comparison and only serves as an accessible educational demo. Please refer to the Claude 3 model card for a more rigorous, quantitative comparison using industry-standard benchmarks

5.5 Picking a model

The next logical question is: which model should you use? It's a difficult question to answer without knowing the specific tasks and demands of a given application. The choice of model can significantly impact the performance, user experience, and cost-effectiveness of your application:

- **Capabilities**
  - The first and foremost consideration is whether the model possesses the necessary capabilities to handle the tasks and use cases specific to your application. Different models have varying levels of performance across different domains, such as general language understanding, task-specific knowledge, reasoning abilities, and generation quality. It's essential to align the model's strengths with the demands of your application to ensure optimal results.

- **Speed**
  - The speed at which a model can process and generate responses is another critical factor, particularly for applications that require real-time or near-real-time interactions. Faster models can provide a more responsive and seamless user experience, reducing latency and improving overall usability. However, it's important to strike a balance between speed and model capabilities, as the fastest model may not always be the most suitable for your specific needs.
- **Cost**
  - The cost associated with using a particular model is a practical consideration that can impact the viability and scalability of your application. Models with higher capabilities often come with a higher price tag, both in terms of API usage costs and computational resources required. It's crucial to assess the cost implications of different models and determine the most cost-effective option that still meets your application's requirements.

5.5.0.1 One approach: start with Haiku

When experimenting, we often recommend starting with the Haiku model. Haiku is a lightweight and fast model that can serve as an excellent starting point for many applications. Its speed and cost-effectiveness make it an attractive option for initial experimentation and prototyping. In many use cases, Haiku proves to be perfectly capable of generating high-quality responses that meet the needs of the application. By starting with Haiku, you can quickly iterate on your application, test different prompts and configurations, and gauge the model's performance without incurring significant costs or latency. If you are unhappy with the responses, it's easy to "upgrade" to a model like Claude 3.5 Sonnet.

5.5.0.2 Evaluating and upgrading

As you develop and refine your application, it's essential to set up a comprehensive suite of evaluations specific to your use case and prompts. These evaluations will serve as a benchmark to measure the performance of your chosen model and help you make informed decisions about potential upgrades.

If you find that Haiku's responses do not meet your application's requirements or if you desire higher levels of sophistication and accuracy, you can easily transition to more capable models like Sonnet or Opus. These models offer enhanced capabilities and can handle more complex tasks and nuanced language understanding.

By establishing a rigorous evaluation framework, you can objectively compare the performance of different models across your specific use case. This empirical evidence will guide your

decision-making process and ensure that you select the model that best aligns with your application's needs.

---

6 Model parameters

6.1 Lesson goals

- Understand the role of the `max_tokens` parameter
- Use the `temperature` parameter to control model responses
- Explain the purpose of `stop_sequence`

As always, let's begin by importing the `anthropic` SDK and loading our API key:

```
from dotenv import load_dotenv
from anthropic import Anthropic

#load environment variable
load_dotenv()

#automatically looks for an "ANTHROPIC_API_KEY" environment variable
client = Anthropic()
```

6.2 Max tokens

There are 3 required parameters that we must include every time we make a request to Claude:

- `model`
- `max_tokens`
- `messages`

So far, we've been using the `max_tokens` parameter in every single request we make, but we haven't stopped to talk about what it is.

Here's the very first request we made:

```
our_first_message = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=500,
    messages=[
        {"role": "user", "content": "Hi there! Please write me a haiku about a pet chicken"}
    ]
)
```

So what is the purpose of `max_tokens`?

6.2.1 Tokens

In short, `max_tokens` controls the maximum number of tokens that Claude should generate in its response. Before we go any further, let's stop for a moment to discuss tokens.

Most Large Language Models don't "think" in full words, but instead work with a series of word-fragments called tokens. Tokens are the small building blocks of a text sequence that Claude processes, understands, and generates texts with. When we provide a prompt to Claude, that prompt is first turned into tokens and passed to the model. The model then begins generating its output **one token at a time**.

For Claude, a token approximately represents 3.5 English characters, though the exact number can vary depending on the language used.

6.2.2 Working with `max_tokens`

The `max_tokens` parameter allows us to set an upper limit on how many tokens Claude generates for us. As an illustration, suppose we ask Claude to write us a poem and set `max_tokens` to 10. Claude will start generating tokens for us and immediately stop as soon as it hits 10 tokens. This will often lead to truncated or incomplete outputs. Let's try it!

```
truncated_response = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=10,
    messages=[
        {"role": "user", "content": "Write me a poem"}
    ]
)
print(truncated_response.content[0].text)
```

```
Here is a poem for you:

The
```

This is what we got back from Claude:

> Here is a poem for you:
>
> The

If you run the above code, you'll likely get a different result that is equally truncated. Claude started to write us a poem and then immediately stopped upon generating 10 tokens.

We can also check the `stop_reason` property on the response Message object to see WHY the model stopped generating. In this case, we can see that it has a value of "max_tokens" which tells us the model stopped generating because it hit our max token limit!

```
truncated_response.stop_reason
```

```
'max_tokens'
```

Of course, if we try generating a poem again with a larger value for `max_tokens`, we'll likely get an entire poem:

```
longer_poem_response = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=500,
    messages=[
        {"role": "user", "content": "Write me a poem"}
    ]
)
print(longer_poem_response.content[0].text)
```

```
Here is a poem for you:

Whispers of the Heart

In the quiet moments,
When the world fades away,
I hear the whispers of my heart -
Gentle words that gently sway.
```

```
They speak of dreams still unfurled,
Of love that shines like the sun,
Of passions yet to be explored,
Of all that's yet to be done.

These whispers, they guide my way,
Reminding me to pause and feel,
To listen closely to the soul,
And let its truths be revealed.

For in the silence, in the calm,
The heart's true voice can be heard,
Weaving a tapestry of hope,
With every softly spoken word.

So I will heed these whispers dear,
And let them be my faithful friend,
For in their song, I find my way,
To all my heart hopes to transcend.
```

This is what Claude generated with `max_tokens` set to 500:

```
Here is a poem for you:

Whispers of the Wind

The wind whispers softly,
Caressing my face with care.
Its gentle touch, a fleeting breath,
Carries thoughts beyond compare.

Rustling leaves dance in rhythm,
Swaying to the breeze's song.
Enchanting melodies of nature,
Peaceful moments linger long.

The wind's embrace, a soothing balm,
Calms the restless soul within.
Embracing life's fleeting moments,
As the wind's sweet song begins.
```

If we look at the `stop_reason` for this response, we'll see a value of "end_turn" which is the

model's way of telling us that it naturally finished generating. It wrote us a poem and had nothing else to say, so it stopped!

```
longer_poem_response.stop_reason
```

```
'end_turn'
```

It's important to note that the models don't "know" about `max_tokens` when generating content. Changing `max_tokens` won't alter how Claude generates the output, it just gives the model room to keep generating (with a high `max_tokens` value) or truncates the output (with a low `max_tokens` value).

It's also important to know that increasing `max_tokens` does not ensure that Claude actually generates a specific number of tokens. If we ask Claude to write a joke and set `max_tokens` to 1000, we'll almost certainly get a response that is much shorter than 1000 tokens.

```
response = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=1000,
    messages=[{"role": "user", "content": "Tell me a joke"}]
)
```

```
print(response.content[0].text)
```

```
Here's a classic dad joke for you:

Why don't scientists trust atoms? Because they make up everything!

How was that? I tried to keep it clean and mildly amusing. Let me know if you'd like to hear
```

```
print(response.usage.output_tokens)
```

```
55
```

In the above example, we ask Claude to "Tell me a joke" and give `max_tokens` a value of 1000. It generated this joke:

```
Here's a classic dad joke for you:

Why don't scientists trust atoms? Because they make up everything!

How was that? I tried to keep it clean and mildly amusing. Let me know if you'd like to hear
```

That generated content was only 55 tokens long. We gave Claude a ceiling of 1000 tokens, but that doesn't mean it will generate 1000 tokens.

6.2.3 Why alter max tokens?

Understanding tokens is crucial when working with Claude, particularly for the following reasons:

- **API limits**: The number of tokens in your input text and the generated response count towards the API usage limits. Each API request has a maximum limit on the number of tokens it can process. Being aware of tokens helps you stay within the API limits and manage your usage efficiently.
- **Performance**: The number of tokens Claude generates directly impacts the processing time and memory usage of the API. Longer input texts and higher max_tokens values require more computational resources. Understanding tokens helps you optimize your API requests for better performance.
- **Response quality**: Setting an appropriate max_tokens value ensures that the generated response is of sufficient length and contains the necessary information. If the max_tokens value is too low, the response may be truncated or incomplete. Experimenting with different max_tokens values can help you find the optimal balance for your specific use case.

Let's take a look at how the number of tokens generated by Claude can impact performance. The following function asks Claude to generate a very long dialogue between two characters three different times, each with a different value for `max_tokens`. It then prints out how many tokens were actually generated and how long the generation took.

```
import time
def compare_num_tokens_speed():
    token_counts = [100,1000,4096]
    task = """
        Create a long, detailed dialogue that is at least 5000 words long between two charact
        The characters should have differing opinions and engage in a respectful thorough del
    """

    for num_tokens in token_counts:
        start_time = time.time()

        response = client.messages.create(
            model="claude-3-haiku-20240307",
            max_tokens=num_tokens,
            messages=[{"role": "user", "content": task}]
```

```
        )

        end_time = time.time()
        execution_time = end_time - start_time

        print(f"Number of tokens generated: {response.usage.output_tokens}")
        print(f"Execution Time: {execution_time:.2f} seconds\n")
```

```
compare_num_tokens_speed()
```

```
Number of tokens generated: 100
Execution Time: 1.51 seconds

Number of tokens generated: 1000
Execution Time: 8.33 seconds

Number of tokens generated: 3433
Execution Time: 28.80 seconds
```

If you run the code, the exact values you get will likely differ, but here's one example output:

```
Number of tokens generated: 100
Execution Time: 1.51 seconds

Number of tokens generated: 1000
Execution Time: 8.33 seconds

Number of tokens generated: 3433
Execution Time: 28.80 seconds
```

As you can see, **the more tokens that Claude generates, the longer it takes!**

For an even more obvious example, we asked Claude to repeat back a very long piece of text and used `max_tokens` to cut off the generation at various output sizes. We repeated this 50 times for each size and calculated the average generation times. As you can see, as the output size grows so does the time it takes! Take a look at the following plot:

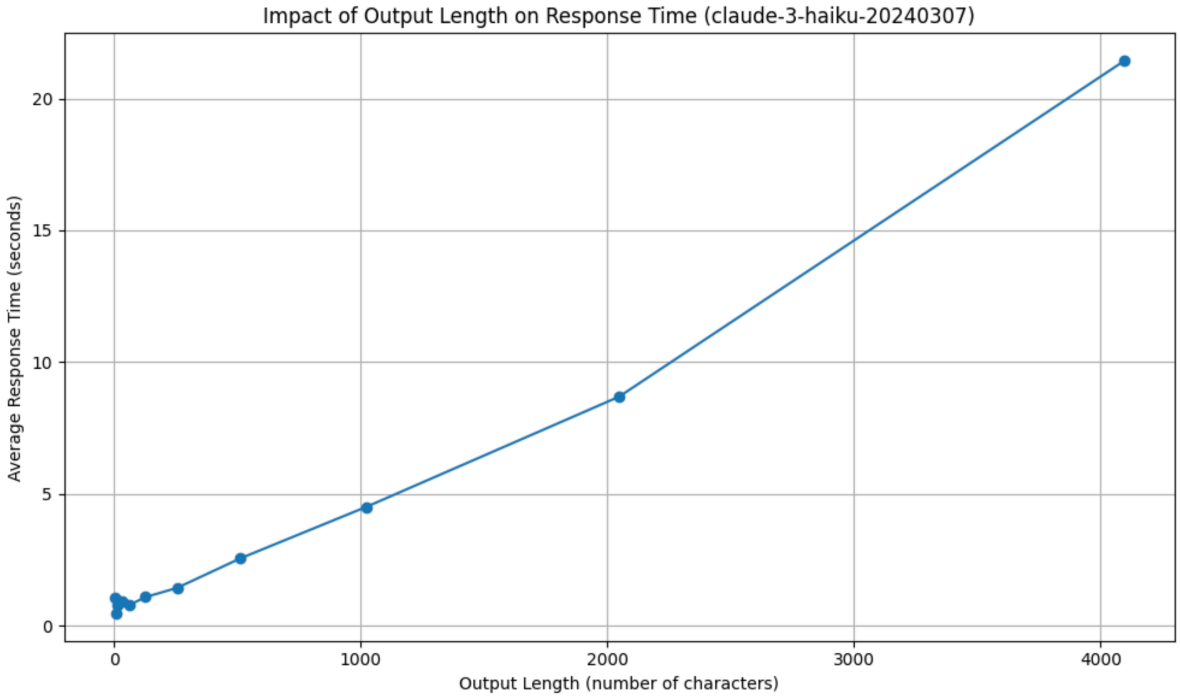

Figure 6.1: output_length.png

6.3 Stop sequences

Another important parameter we haven't seen yet is `stop_sequence` which allows us to provide the model with a set of strings that, when encountered in the generated response, cause the generation to stop. They are essentially a way of telling Claude, "if you generate this sequence, stop generating anything else!"

Here's an example of a request that does not include a `stop_sequence`:

```
response = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=500,
    messages=[{"role": "user", "content": "Generate a JSON object representing a person with
)
print(response.content[0].text)
```

```
Here's an example of a JSON object representing a person with a name, email, and phone numbe

```json
```
```

```
{
  "name": "John Doe",
  "email": "johndoe@example.com",
  "phoneNumber": "123-456-7890"
}
```

In this example, the JSON object has three key-value pairs:

1. "name": The person's name, which is a string value of "John Doe".
2. "email": The person's email address, which is a string value of "johndoe@example.com".
3. "phoneNumber": The person's phone number, which is a string value of "123-456-7890".

You can modify the values to represent a different person with their own name, email, and pho

The above code asks Claude to generate a JSON object representing a person. Here's an example output Claude generated:

Here's an example of a JSON object representing a person with a name, email, and phone numbe

```
{
  "name": "John Doe",
  "email": "johndoe@example.com",
  "phoneNumber": "123-456-7890"
}
```

In this example, the JSON object has three key-value pairs:

1. "name": The person's name, which is a string value of "John Doe".
2. "email": The person's email address, which is a string value of "johndoe@example.com".
3. "phoneNumber": The person's phone number, which is a string value of "123-456-7890".

You can modify the values to represent a different person with their own name, email, and pho

Claude did generate the requested object, but also included an explanation afterwards. If we wanted Claude to stop generating as soon as it generated the closing "}" of the JSON object, we could modify the code to include the `stop_sequences` parameter.

```
response = client.messages.create(
    model="claude-3-haiku-20240307",
```

```
    max_tokens=500,
    messages=[{"role": "user", "content": "Generate a JSON object representing a person with
    stop_sequences=["}"]
)
print(response.content[0].text)
```

```
Here's a JSON object representing a person with a name, email, and phone number:

{
  "name": "John Doe",
  "email": "john.doe@example.com",
  "phone": "555-1234"
```

The model generated the following output:

```
Here's a JSON object representing a person with a name, email, and phone number:
{
  "name": "John Doe",
  "email": "john.doe@example.com",
  "phone": "555-1234"
```

IMPORTANT NOTE: Notice that the resulting output does **not** include the "}" stop sequence itself. If we wanted to use and parse this as JSON, we would need to add the closing "}" back in.

When we get a response back from Claude, we can check why the model stopped generating text by inspecting the `stop_reason` property. As you can see below, the previous response stopped because of 'stop_sequence' which means that the model generated one of the stop sequences we provided and immediately stopped.

```
response.stop_reason
```

```
'stop_sequence'
```

We can also look at the `stop_sequence` property on the response to check which particular stop_sequence caused the model to stop generating:

```
response.stop_sequence
```

```
'}'
```

We can provide multiple stop sequences. In the event that we provide multiple, the model will stop generating as soon as it encounters any of the stop sequences. The resulting `stop_sequence` property on the response Message will tell us which exact `stop_sequence` was encountered.

The function below asks Claude to write a poem and stop if it ever generates the letters "b" or "c". It does this three times:

```
def generate_random_letters_3_times():
    for i in range(3):
        response = client.messages.create(
            model="claude-3-haiku-20240307",
            max_tokens=500,
            messages=[{"role": "user", "content": "generate a poem"}],
            stop_sequences=["b", "c"]
        )
        print(f"Response {i+1} stopped because {response.stop_reason}.  The stop sequence was
```

```
generate_random_letters_3_times()
```

```
Response 1 stopped because stop_sequence.  The stop sequence was c
Response 2 stopped because stop_sequence.  The stop sequence was b
Response 3 stopped because stop_sequence.  The stop sequence was b
```

Here's an example output:

```
Response 1 stopped because stop_sequence.  The stop sequence was c
Response 2 stopped because stop_sequence.  The stop sequence was b
Response 3 stopped because stop_sequence.  The stop sequence was b
```

The first time through, Claude stopped writing the poem because it generated the letter "c". The following two times, it stopped because it generated the letter "b". Would you ever do this? Probably not!

6.4 Temperature

The `temperature` parameter is used to control the "randomness" and "creativity" of the generated responses. It ranges from 0 to 1, with higher values resulting in more diverse and unpredictable responses with variations in phrasing. Lower temperatures can result in more

deterministic outputs that stick to the most probable phrasing and answers. **Temperature has a default value of 1**.

When generating text, Claude predicts the probability distribution of the next token (word or subword). The temperature parameter is used to manipulate this probability distribution before sampling the next token. If the temperature is low (close to 0.0), the probability distribution becomes more peaked, with high probabilities assigned to the most likely tokens. This makes the model more deterministic and focused on the most probable or "safe" choices. If the temperature is high (closer to 1.0), the probability distribution becomes more flattened, with the probabilities of less likely tokens increasing. This makes the model more random and exploratory, allowing for more diverse and creative outputs.

See this diagram for a visual representation of the impact of temperature:

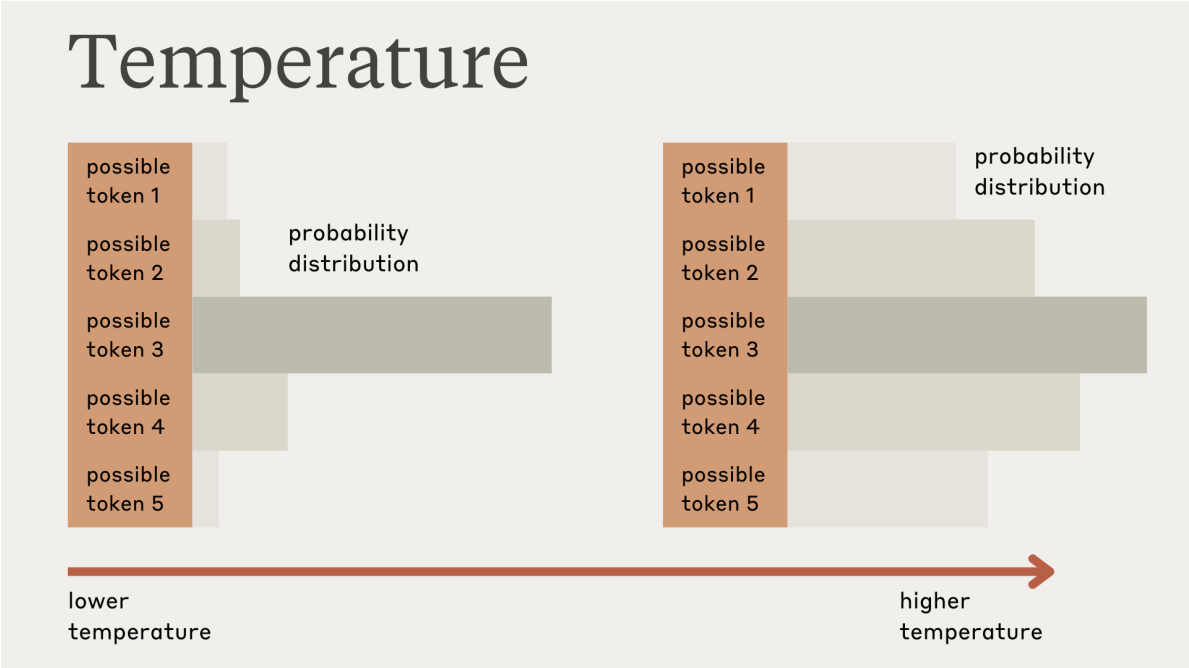

Figure 6.2: temperature.png

Why would you change temperature?

Use temperature closer to 0.0 for analytical tasks, and closer to 1.0 for creative and generative tasks.

Let's try a quick demonstration. Take a look at the function below. Using a temperature of 0 and then a temperature of 1, we make three requests to Claude, asking it to "Come up with a name for an alien planet. Respond with a single word."

```python
def demonstrate_temperature():
    temperatures = [0, 1]
    for temperature in temperatures:
        print(f"Prompting Claude three times with temperature of {temperature}")
        print("================")
        for i in range(3):
            response = client.messages.create(
                model="claude-3-haiku-20240307",
                max_tokens=100,
                messages=[{"role": "user", "content": "Come up with a name for an alien plan
                temperature=temperature
            )
            print(f"Response {i+1}: {response.content[0].text}")
```

```python
demonstrate_temperature()
```

```
Prompting Claude three times with temperature of 0
================
Response 1: Xendor.
Response 2: Xendor.
Response 3: Xendor.
Prompting Claude three times with temperature of 1
================
Response 1: Xyron.
Response 2: Xandar.
Response 3: Zyrcon.
```

This is the result of running the above function (your specific results may vary):

```
Prompting Claude three times with temperature of 0
================
Response 1: Xendor.
Response 2: Xendor.
Response 3: Xendor.
Prompting Claude three times with temperature of 1
================
Response 1: Xyron.
Response 2: Xandar.
Response 3: Zyrcon.
```

Notice that with a temperature of 0, all three responses are the same. Note that even with a temperature of 0.0, the results will not be fully deterministic. However, there is a clear difference when compared to the results with a temperature of 1. Each response was a completely different alien planet name.

Below is a chart that illustrates the impact temperature can have on Claude's outputs. Using this prompt, "Pick any animal in the world. Respond with only a single word: the name of the animal," we queried Claude 100 times with a temperature of 0. We then did it 100 more times, but with a temperature of 1. The plot below shows the frequencies of each animal response Claude came up with.

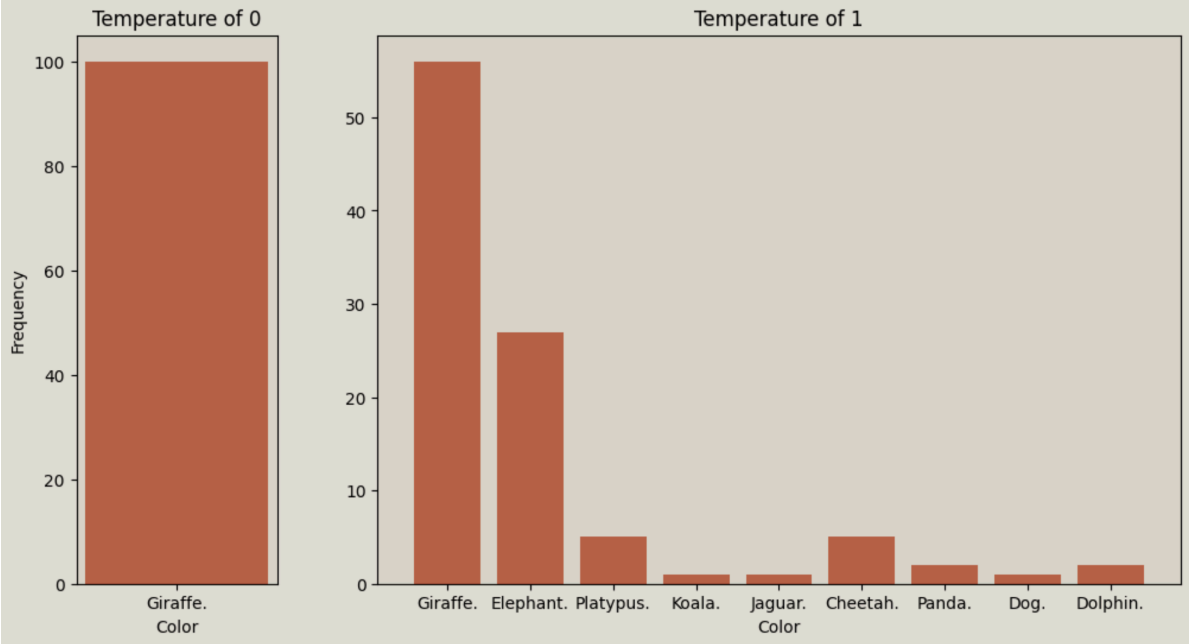

Figure 6.3: temperature_plot.png

As you can see, with a temperature of 0, Claude responded with "Giraffe" every single time. Please remember that a temperature of 0 does not guarantee deterministic results, but it does make Claude much more likely to respond with similar content each time. With a temperature of 1, Claude still chose giraffe more than half the time, but the responses also include many other types of animals!

6.5 System prompt

The `system_prompt` is an optional parameter you can include when sending messages to Claude. It sets the stage for the conversation by giving Claude high-level instructions, defining its role,

or providing background information that should inform its responses.

Key points about the system_prompt:

- It's optional but can be useful for setting the tone and context of the conversation.
- It's applied at the conversation level, affecting all of Claude's responses in that exchange.
- It can help steer Claude's behavior without needing to include instructions in every user message.

Note that for the most part, only tone, context, and role content should go inside the system prompt. Detailed instructions, external input content (such as documents), and examples should go inside the first `User` turn for better results. You do not need to repeat this for every subsequent `User` turn.

Let's try it out:

```
message = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=1000,
    system="You are a helpful foreign language tutor that always responds in French.",
    messages=[
        {"role": "user", "content": "Hey there, how are you?!"}
    ]
)

print(message.content[0].text)
```

```
Bonjour ! Je suis ravi de vous rencontrer. Comment allez-vous aujourd'hui ?
```

6.6 Exercise

Write a function called `generate_questions` that does the following: * Takes two parameters: `topic` and `num_questions` * Generates `num_questions` thought-provoking questions about the provided `topic` as a numbered list * Prints the generated questions

For example, calling `generate_questions(topic="free will", num_questions=3)` could result in the following output:

1. To what extent do our decisions and actions truly originate from our own free will, rather than being shaped by factors beyond our control, such as our genes, upbringing, and societal influences?

2. If our decisions are ultimately the result of a complex interplay of biological, psychological, and environmental factors, does that mean we lack the ability to make authentic, autonomous choices, or is free will compatible with determinism?
3. What are the ethical and philosophical implications of embracing or rejecting the concept of free will? How might our views on free will impact our notions of moral responsibility, punishment, and the nature of the human condition?

In your implementation, please make use of the following parameters: * `max_tokens` to limit the response to under 1000 tokens * `system` to provide a system prompt telling the model it is an expert on the particular `topic` and should generate a numbered list. * `stop_sequences` to ensure the model stops after generating the correct number of questions. (If we ask for 3 questions, we want to make sure the model stops as soon as it generates "4." If we ask for 5 questions, we want to make sure the model stops as soon as it generates "6.")

6.6.0.1 Potential solution

```
def generate_questions(topic, num_questions=3):
    response = client.messages.create(
        model="claude-3-haiku-20240307",
        max_tokens=500,
        system=f"You are an expert on {topic}. Generate thought-provoking questions about thi
        messages=[
            {"role": "user", "content": f"Generate {num_questions} questions about {topic} as
        ],
        stop_sequences=[f"{num_questions+1}."]
    )
    print(response.content[0].text)
```

```
generate_questions(topic="free will", num_questions=3)
```

```
Here are three thought-provoking questions about free will:

1. To what extent do our decisions and actions truly originate from our own free will, rather

2. If our decisions are ultimately the result of a complex interplay of biological, psycholog

3. What are the ethical and philosophical implications of embracing or rejecting the concept
```

---

7 Streaming

7.1 Learning goals

- Understand how streaming works
- Work with stream events

Let's start by importing the `anthropic` SDK and setting up our client:

```
from dotenv import load_dotenv
from anthropic import Anthropic

#load environment variable
load_dotenv()

#automatically looks for an "ANTHROPIC_API_KEY" environment variable
client = Anthropic()
```

So far, we've sent messages to Claude using the following syntax:

```
response = client.messages.create(
    messages=[
        {
            "role": "user",
            "content": "Write me an essay about macaws and clay licks in the Amazon",
        }
    ],
    model="claude-3-haiku-20240307",
    max_tokens=800,
    temperature=0,
)
print("We have a response back!")
print("========================")
print(response.content[0].text)
```

```
We have a response back!
========================
Here is an essay about macaws and clay licks in the Amazon:

Macaws and Clay Licks in the Amazon

Deep within the lush, verdant rainforests of the Amazon basin, a remarkable natural phenomen

Macaws are some of the most striking and iconic birds of the Amazon. With their vividly-hued

These magnificent birds play a crucial role in the Amazon ecosystem, acting as important seed

Clay licks are exposed deposits of mineral-rich clay that attract hundreds, sometimes thousan

Visiting a clay lick is a truly awe-inspiring experience. As the first rays of dawn break thr

The importance of clay licks to the survival of macaws and other Amazonian wildlife cannot be
```

This works fine, but it's important to remember that with this approach, we only get content back from the API **once all of the content has been generated.** Try running the above cell again, and you'll see that nothing is printed out until the entire response is printed all at once. This is fine in many situations, but it can lead to bad user experiences if you're building an application that forces a user to wait around for an entire response to be generated.

Enter streaming!

Streaming enables us to write applications that receive content as it is generated by the model, rather than having to wait for the entire response to be generated. This is how apps like claude.ai work. As the model generates responses, that content is streamed to a user's browser and displayed:

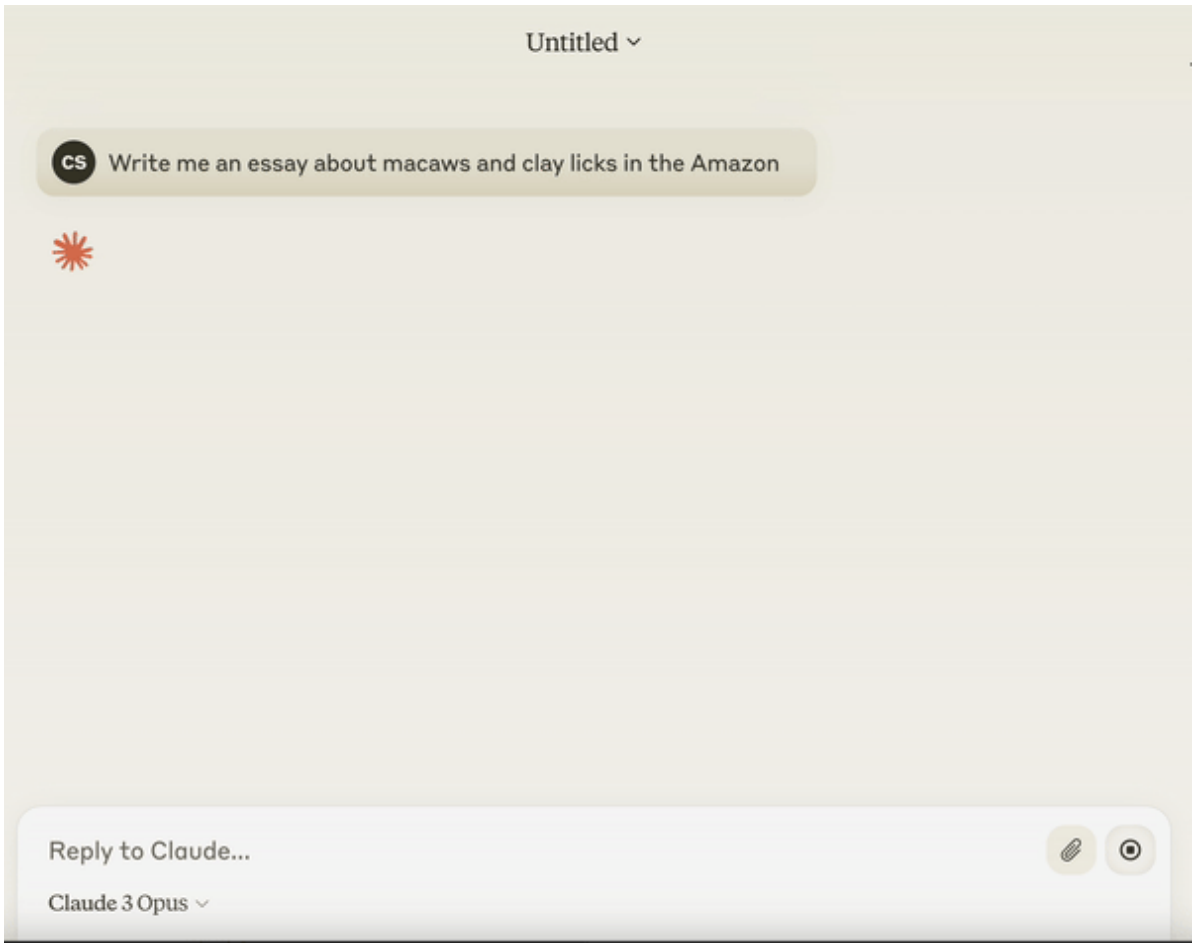

Figure 7.1: Streaming Demo

7.2 Working with streams

To get a streaming response from the API, it's as simple as passing `stream=True` to `client.messages.create`. That's the easy part. Where things get a little trickier is in how we subsequently work with the streaming response and handle incoming data.

```
stream = client.messages.create(
    messages=[
        {
            "role": "user",
            "content": "Write me a 3 word sentence, without a preamble.  Just give me 3 words
        }
```

```
    ],
    model="claude-3-haiku-20240307",
    max_tokens=100,
    temperature=0,
    stream=True,
)
```

Let's take a look at our `stream` variable

```
stream
```

```
<anthropic.Stream at 0x111e5ec10>
```

There's not much to look at! This stream object on its own won't do a whole lot for us. The stream object is a generator object that yields individual server-sent events (SSE) as they are received from the API. We need to write code to iterate over it and work with each individual server-sent event. Remember, our data is no longer coming in one finalized chunk of data. Let's try iterating over the stream response:

```
for event in stream:
    print(event)
```

```
MessageStartEvent(message=Message(id='msg_01EZHjA6qmf6y8VWMZSeBmN5', content=[], model='claud
ContentBlockStartEvent(content_block=ContentBlock(text='', type='text'), index=0, type='conte
ContentBlockDeltaEvent(delta=TextDelta(text='Cats', type='text_delta'), index=0, type='conten
ContentBlockDeltaEvent(delta=TextDelta(text=' me', type='text_delta'), index=0, type='content
ContentBlockDeltaEvent(delta=TextDelta(text='ow lou', type='text_delta'), index=0, type='cont
ContentBlockDeltaEvent(delta=TextDelta(text='dly.', type='text_delta'), index=0, type='conten
ContentBlockStopEvent(index=0, type='content_block_stop')
MessageDeltaEvent(delta=Delta(stop_reason='end_turn', stop_sequence=None), type='message_delt
MessageStopEvent(type='message_stop')
```

As you can see, we ended up receiving many server-sent events from the API. Let's take a closer look at what these events mean. This is a color-coded explanation of the events:

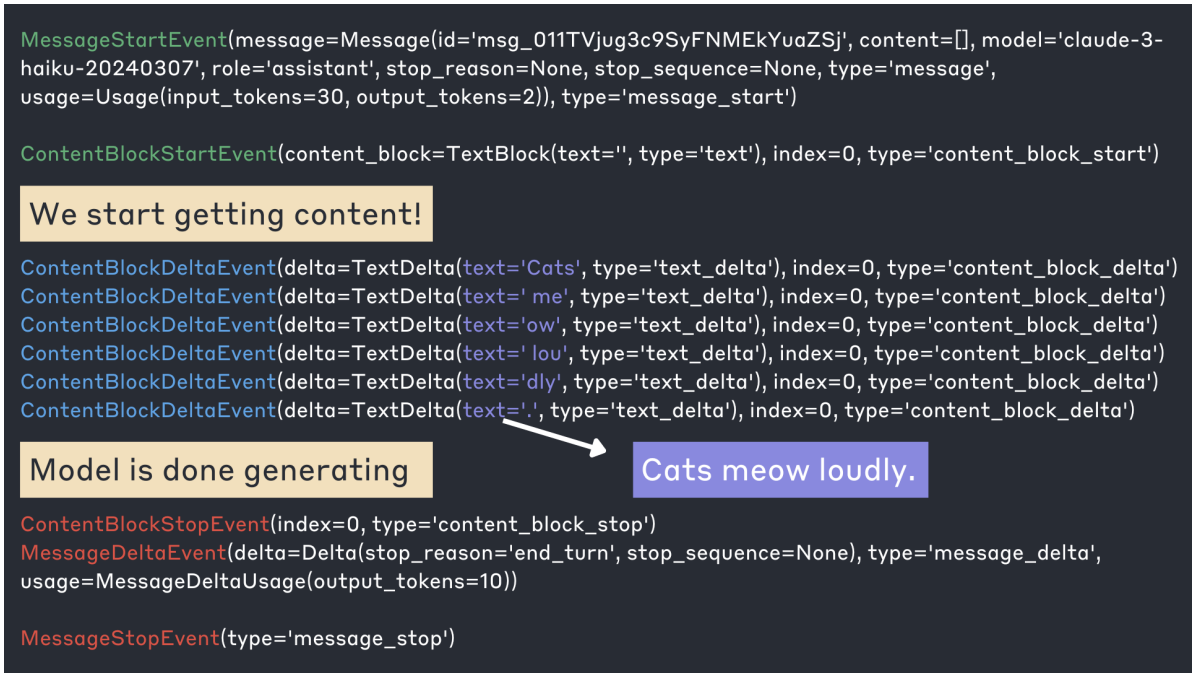

Figure 7.2: streaming_output.png

Each stream contains a series of events in the following order: * **MessageStartEvent** - A message with empty content * **Series of content blocks** - Each of which contains: * A **ContentBlockStartEvent** * One or more **ContentBlockDeltaEvent**s * A **ContentBlockStopEvent** * One or more **MessageDeltaEvent**s which indicate top-level changes to the final message * A final **MessageStopEvent**

In the above response, there was only a single content block. This diagram shows all the events associated with it:

Figure 7.3: content_block_streaming.png

All of the actual model-generated content we care about comes from the ContentBlockDeltaEvents, which each contain a type set to "content_block_delta." To actually get the content itself, we want to access the `text` property inside of `delta`. Let's try exclusively printing out the text that was generated:

```
stream = client.messages.create(
    messages=[
        {
            "role": "user",
            "content": "Write me a 3 word sentence, without a preamble.  Just give me 3 words
        }
    ],
    model="claude-3-haiku-20240307",
    max_tokens=100,
    temperature=0,
    stream=True,
)
for event in stream:
    if event.type == "content_block_delta":
        print(event.delta.text)
```